基于香橙派鲲鹏Pro开发板与openGauss的公交安全管理系统实验

建议学时：16学时

一、 实验任务

1. 任务描述

香橙派鲲鹏Pro是一款基于华为鲲鹏处理器的国产高性能开发板，适合部署边缘应用。openGauss则是华为开源的高性能、高安全企业级关系型数据库，支持国产化生态。本实验将指导学生基于该开发板与openGauss，构建一个完整的包含后台数据库、数据库访问程序以及前台网页的公交安全管理系统。实验涵盖从运用规范化理论设计数据库模式、实现完整性约束与索引优化，到借助大模型辅助生成前后端代码、最终在香橙派开发板上部署完整系统的全流程。通过本任务，学生将掌握在国产化软硬件平台上进行数据库系统全栈设计与实现的能力，重点在于深入理解数据库规范化设计、完整性约束与性能优化，并实践利用AI工具提升开发效率。

1) 实现 openGauss 后台数据库

数据库的有关语义如下：

(1) 公交公司有若干个车队，每个车队下有若干条线路；

(2) 公交公司有若干辆汽车，每辆车属于一条线路；

(3) 每个车队有一名队长，他只有管理工作，不开车；

(4) 每条线路有若干名司机，其中有一名路队长，除开车外，还承担管理工作；每名司机只在一条线路上开车；

(5) 司机开车时会产生违章，包含：闯红灯、未礼让斑马线、压线、违章停车等；

(6) 队长、路队长负责将司机的违章信息输入到系统，包含：司机、车辆、车队、线路、站点、时间、违章等。

2) 前台网页

开发一个公交安全管理系统来对数据库进行访问，可以使用Java、Python、C等集成开发环境，功能如下：

(1) 录入司机基本信息，如工号、姓名、性别等；

(2) 录入汽车基本信息，如车牌号、座数等；

23009200798 乔舟

(3) 录入司机的违章信息；

(4) 查询某个车队下的司机基本信息；

(5) 查询某名司机在某个时间段的违章详细信息；

(6) 查询某个车队在某个时间段的违章统计信息，如：2次闯红灯、4次未礼让斑马线等。

3) 注意事项

(1) 后台数据库和前台程序必须使用大模型，如千问，豆包、deepseek、chatgpt、gemini等；

(2) 在数据库的设计过程中需要运用规范化理论，避免出现插入异常、删除异常、数据冗余等问题；

(3) 必须设定关系的完整性规则，如实体完整性（例如设置主码），参照完整性（例如设置外码和对应的主码），用户自定义完整性（例如性别只能为“男”或“女”）；

(4) 使用索引来加快查询的速度；使用视图、函数、存储过程来简化系统的设计；

(5) 使用数据库设计工具，例如：ERWin，PowerDesigner;

(6) 实习重点在于后台数据库的设计，对于前台程序的开发，能够实现系统功能即可，不要把大量时间花费在界面设计和不必要的代码上。

2. 学习目标

完成本任务的学习后，你应当能：

1) 理解关系数据库规范化理论，并能应用于设计无冗余、低耦合的数据库模式；

2) 掌握在 openGauss 数据库中定义实体完整性、参照完整性与用户自定义完整性的方法；

3) 能够熟练使用数据库设计工具（如 ERWin、PowerDesigner）进行概念模型与物理模型设计；

4) 掌握运用 SQL 语句创建索引、视图、存储过程及函数以优化查询与简化应用逻辑；

5) 理解并实践在香橙派鲲鹏 Pro 国产开发板上部署 openGauss 数据库及 Web 应用的全过程；

6) 能够利用大模型辅助生成符合业务需求的数据操作与前后端基础代码；

7) 掌握一个完整数据库应用系统的开发流程，从前端界面交互到后端业务逻辑与数据库访问；

8) 具备根据业务语义进行数据查询、统计与分析，并解决实际管理问题的能力。

23009200798 何沐书

二、 任务准备

1. 前置知识

本实验案例需要提前学习关系数据库系统、SQL基础、数据库规范化设计理论、openGauss数据库、香橙派鲲鹏Pro开发板、后端与前端全栈开发基础、Linux操作以及数据库应用系统开发流程。可通过如下途径进行学习：

1) 关系数据库系统基础

理解数据库基本概念，如实体、属性、关系、模式，熟悉关系模型和SQL的常用语法，能够使用openGauss数据库创建数据库、表，进行基本的增删改查操作。

2) 数据库设计理论

了解概念模型E-R图的绘制方法，掌握数据库规范化理论中一范式、二范式、三范式的核心思想及其作用，能够使用ERWin、PowerDesigner等工具设计数据库。

3) 香橙派鲲鹏Pro开发板

了解开发板硬件接口GPIO、网口，基于ARM架构的openEuler操作系统，网络配置及远程登录SSH，掌握通过命令行进行软件包管理与基本运维。

4) 后端编程

掌握后端编程语言Python或Java的基础语法，了解连接和操作数据库的基本方法JDBC、Psycopg2等驱动或框架。

5) 前端开发

具备简单的网页开发知识，了解HTML、CSS及JavaScript的基础，或掌握基本Web框架Flask、Spring Boot用于构建简单的数据交互界面。

6) 大模型辅助开发

掌握大模型DeepSeek、ChatGPT、Gemni、GraphRAG等工具。在数据库设计阶段辅助生成E-R图、SQL的DDL、DML语句，并按照规范化理论优化数据库。在后端和前端开发阶段生成后端数据库连接、数据操作的核心代码框架，提供前端页面的基础HTML/JavaScript代码。在系统调试与优化阶段解释SQL错误、分析查询性能，并对现有代码进行注释、重构或优化建议。

2. 实验环境准备

本实验需要在香橙派鲲鹏Pro开发板及本地计算机上搭建完整的软硬件开发环境。香橙派鲲鹏Pro开发板作为国产高性能嵌入式平台，为系统后端提供了稳定的运行基础；本地计算机则用于数据库设计与前端开发。本实验环境准备主要包括香橙派鲲鹏Pro开发板的网络

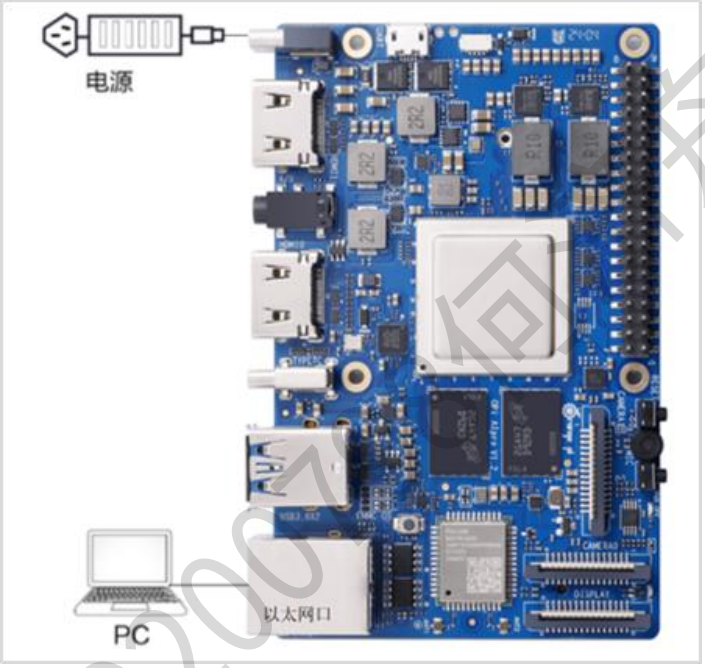
23009200798何济舟

配置、openGauss数据库的安装与初始化、以及前端开发工具链（如Python/Java环境、数据库连接驱动、Web框架）的配置。通过系统的环境搭建，确保后续的数据库设计、应用编程与系统集成能够在稳定可靠的网络环境中顺利进行。

使用Docker容器技术启动香橙派鲲鹏Pro开发板上的openGauss数据库，并通过Pycharm插件连接到本地电脑。

1) 连接并登录到香橙派鲲鹏Pro开发板：

- 网线直连开发板



使用网线连接开发板和PC。开发板插入网线，另一端连接PC，启动上电。

- 配置个人PC的网络设置

开发板通过网线和个人PC连接后，需要使开发板的ip地址（192.168.10.8）和个人PC的以太网处于同一网段，才能实现网络互联。具体操作步骤如下：

(1) 打开更改适配器选项

选择“网络和Internet”，单击“更改适配器选项”：

(2) 打开连接网线对应的以太网属性

出现已连接的“以太网x”就是开发板网线连接接口，如下图中的“以太网3”，选中后右键打开，点击“以太网x”的属性：

(3) 修改IPV4属性

勾选“使用下面的IP地址”选项，填写IP地址（此处设置为192.168.10.1）和子网掩码，默认网关与DNS服务器地址为空，单击“确定”保存：

2300920798何济舟

(4) 验证连接是否成功

23009200798何济舟

在键盘上使用快捷键“Win+R”，在运行窗口输入cmd进入命令行窗口。输入ipconfig命令查询PC网口IP地址是否修改成功：

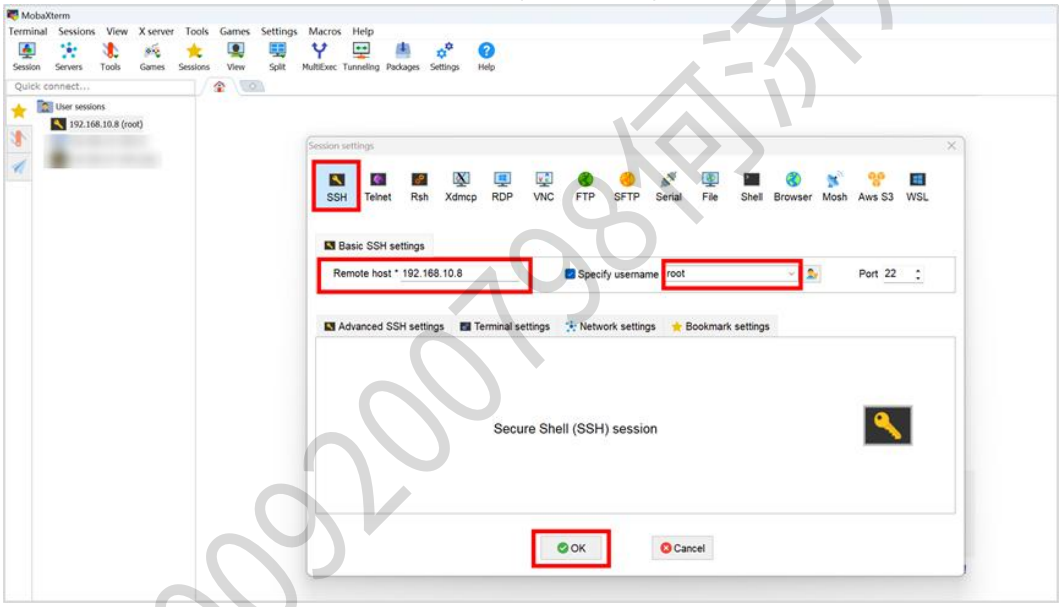
```
以太网适配器 以太网 3:

   连接特定的 DNS 后缀 . . . . . . . :
   IPv4 地址 . . . . . . . . . . . . : 192.168.1.[illegible]
   子网掩码 . . . . . . . . . . . . : 255.255.255.0
   默认网关. . . . . . . . . . . . . :
```

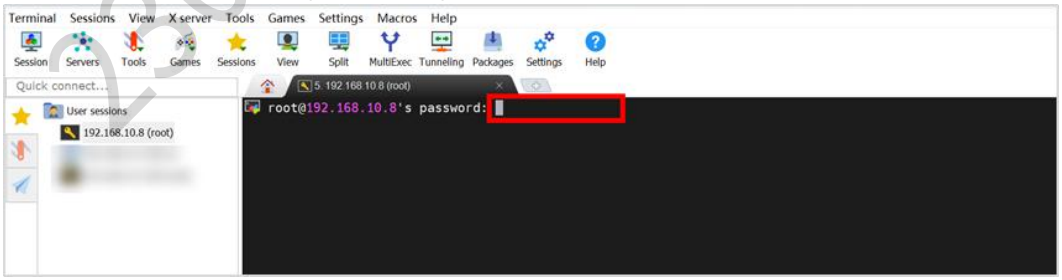
- 使用ssh工具登录开发板

可以通过其它ssh工具软件登录到开发板，如MobaXtrem、xshell、putty等。这里以MobaXtrem为例显示。

从本地电脑通过配置香橙派鲲鹏Pro开发板的IP地址（如：192.168.10.8）来连接香橙派鲲鹏Pro开发板，并使用root用户及对应密码(openEuler)来登录。



输入root用户及对应密码(openEuler)，然后按“Enter”键登录。



- 开发板网络连接（WIFI连接）

(1) 开发板上执行nmcli dev wifi命令扫描周围的WIFI热点：

2200920079 8何济丹

```
[root@kp-pro ~]#
[root@kp-pro ~]# nmcli dev wifi
IN-USE  BSSID              SSID             MODE   CHAN  RATE        SIGNAL  BARS  SECURITY
        7C:C3:85:BE:33:E1  Huawei-Employee  Infra  9     260 Mbit/s  94      ▂▄▆▇  WPA2 802.1X
*       7C:C3:85:BE:33:F2  Huawei-Guest     Infra  149   260 Mbit/s  85      ▂▄▆▇  --
        7C:C3:85:BE:33:F7  --               Infra  149   260 Mbit/s  85      ▂▄▆▇  WPA1 WPA2
        7C:C3:85:BE:33:F1  Huawei-Employee  Infra  149   260 Mbit/s  82      ▂▄▆▇  WPA2 802.1X
        D2:8C:D9:D4:89:AF  Ljm_iPhone       Infra  6     130 Mbit/s  80      ▂▄▆_  WPA2
        7C:C3:85:BE:33:F0  wlanaccessv2.0   Infra  149   260 Mbit/s  80      ▂▄▆_  WPA2 802.1X
        7C:C3:85:BE:30:70  wlanaccessv2.0   Infra  40    260 Mbit/s  57      ▂▄▆_  WPA2 802.1X
        7C:C3:85:BE:30:71  Huawei-Employee  Infra  40    260 Mbit/s  57      ▂▄▆_  WPA2 802.1X
        7C:C3:85:BE:30:72  Huawei-Guest     Infra  40    260 Mbit/s  57      ▂▄▆_  --
        7C:C3:85:BE:33:A1  Huawei-Employee  Infra  5     260 Mbit/s  50      ▂▄__  WPA2 802.1X
        7C:C3:85:BE:30:61  Huawei-Employee  Infra  5     260 Mbit/s  50      ▂▄__  WPA2 802.1X
        7C:C3:85:BE:33:B7  --               Infra  157   260 Mbit/s  49      ▂▄__  WPA1 WPA2
        AC:75:1D:F6:09:72  Huawei-Guest     Infra  48    260 Mbit/s  47      ▂▄__  --
        7C:C3:85:BE:33:B0  wlanaccessv2.0   Infra  157   260 Mbit/s  47      ▂▄__  WPA2 802.1X
        7C:C3:85:BE:33:B1  Huawei-Employee  Infra  157   260 Mbit/s  47      ▂▄__  WPA2 802.1X
        7C:C3:85:BE:33:B2  Huawei-Guest     Infra  157   260 Mbit/s  47      ▂▄__  --
        AC:75:1D:F6:09:70  wlanaccessv2.0   Infra  48    260 Mbit/s  45      ▂▄__  WPA2 802.1X
        AC:75:1D:F6:09:71  Huawei-Employee  Infra  48    260 Mbit/s  45      ▂▄__  WPA2 802.1X
        AC:75:1D:F6:09:77  --               Infra  48    260 Mbit/s  44      ▂▄__  WPA1 WPA2
        AC:75:1D:F5:F0:F7  --               Infra  153   260 Mbit/s  44      ▂▄__  WPA1 WPA2
        7C:C3:85:BE:33:11  Huawei-Employee  Infra  165   260 Mbit/s  44      ▂▄__  WPA2 802.1X
        7C:C3:85:BE:33:12  Huawei-Guest     Infra  165   260 Mbit/s  44      ▂▄__  --
        AC:75:1D:F5:F0:F0  wlanaccessv2.0   Infra  153   260 Mbit/s  42      ▂▄__  WPA2 802.1X
        AC:75:1D:F5:F0:F1  Huawei-Employee  Infra  153   260 Mbit/s  42      ▂▄__  WPA2 802.1X
        AC:75:1D:F5:F0:F2  Huawei-Guest     Infra  153   260 Mbit/s  42      ▂▄__  --
        7C:C3:85:BE:33:10  wlanaccessv2.0   Infra  165   260 Mbit/s  40      ▂▄__  WPA2 802.1X
```

(2) 使用nmcli命令连接扫描到的WIFI热点，其中wifi_name需要换成想连接的WIFI热点的名字。

wifi_passwd需要换成想连接的WIFI热点的密码。

nmcli dev wifi connect wifi_name password wifi_passwd

连接成功后输出：成功用"wlan0ab877337-e465-440a-852d-c95ee6b5f4b0"激活了设备。

(3) 使用ping命令可以测试WIFI网络的连通性，ping命令可以通过Ctrl+C快捷键来中断运行。

ping www.hikunpeng.com -I wlan0

注意事项：校园网连接上之后无法ping通，可通过手机热点连接。

```
[root@kp-pro ~]# ping www.hikunpeng.com -I wlan0
PING hcdnwsa.global.c.cdnhwc6.com (114.230.213.76) 来自 7.249.233.43 wlan0 56(84) 字节的数据。
64 字节，来自 114.230.213.76 (114.230.213.76): icmp_seq=1 ttl=49 时间=29.9 毫秒
64 字节，来自 114.230.213.76 (114.230.213.76): icmp_seq=2 ttl=49 时间=28.8 毫秒
64 字节，来自 114.230.213.76 (114.230.213.76): icmp_seq=3 ttl=49 时间=27.3 毫秒
64 字节，来自 114.230.213.76 (114.230.213.76): icmp_seq=4 ttl=49 时间=28.9 毫秒
64 字节，来自 114.230.213.76 (114.230.213.76): icmp_seq=5 ttl=49 时间=28.2 毫秒
64 字节，来自 114.230.213.76 (114.230.213.76): icmp_seq=6 ttl=49 时间=28.5 毫秒
^C
--- hcdnwsa.global.c.cdnhwc6.com ping 统计 ---
已发送 6 个包， 已接收 6 个包，0% packet loss, time 5007ms
rtt min/avg/max/mdev = 27.257/28.608/29.949/0.803 ms
[root@kp-pro ~]#
```

2) docker容器启动openGauss：

删除防火墙（删除后开发板需要重启登录）

`yum remove firewalld` //删除防火墙

`systemctl status iptables` //查看路由状态

`systemctl status firewalld` //查看火墙状态

- 下载

docker-compose<https://github.com/docker/compose/releases>

- 创建工作目录

`mkdir /usr/local/dbsy` 将下载的文件 docker-compose-linux-aarch64 上传至 /usr/local/dbsy目录。

- 安装docker-compose

[root@openEuler docker]# cd /usr/local/dbsy

[root@openEuler dbsy]# mv docker-compose-linux-aarch64 /usr/bin/docker-compose

[root@openEuler dbsy]# chmod 777 /usr/bin/docker-compose

[root@openEuler dbsy]# cd /usr/local/dbsy/dbsy-1.1.3/docker

[root@openEuler docker]# yum install docker

[root@openEuler docker]# systemctl start docker

- 拉取镜像

拉取(这一步会需要一点时间)

```
docker pull swr.cn-north-4.myhuaweicloud.com/kunpeng-ai/opengauss:7.0.0-RC1
```

改名

```
docker tag swr.cn-north-4.myhuaweicloud.com/kunpeng-ai/opengauss:7.0.0-RC1
opengauss/opengauss:7.0.0-RC1
```

- 启动容器

```
docker run --name gaussdb \
--privileged=true \
-d \
-p 5432:5432 \
-e GS_PASSWORD="Gauss@1234" \
opengauss/opengauss:7.0.0-RC1
```

容器第一次启动要连接数据库等，所以会花费几分钟（注：关闭容器命令 docker-compose down）。一切顺利会得到类似这样结果：

23009200798何济舟

```
[root@localhost docker]# docker ps |grep docker-
0979a2145c92   nginx:latest                                   "sh -c 'cp /docker-e..."   24 seconds ago   Up
12 seconds   0.0.0.0:80->80/tcp, :::80->80/tcp, 0.0.0.0:443->443/tcp, :::443->443/tcp   docker-nginx-
1
d93e1f593e07   langgenius/dify-api:1.1.3                      "/bin/bash /entrypoi..."   27 seconds ago   Up
12 seconds   5001/tcp
docker-api-1
27820a2bc93d   langgenius/dify-api:1.1.3                      "/bin/bash /entrypoi..."   27 seconds ago   Up
12 seconds   5001/tcp
docker-worker-1
f38346f15def   langgenius/dify-plugin-daemon:0.0.5-local      "/bin/bash -c /app/e..."   27 seconds ago   Up
12 seconds   0.0.0.0:5003->5003/tcp, :::5003->5003/tcp   docker-
plugin_daemon-1
9f79d4824c2b   opengauss/opengauss:7.0.0-RC1                  "entrypoint.sh gauss..."   27 seconds ago
Up 12 seconds (healthy)   5432/tcp, 0.0.0.0:6600->6600/tcp, :::6600->6600/tcp
docker-opengauss-1
a1a026ba82ef   langgenius/dify-web:1.1.3                      "/bin/sh ./entrypoin..."   9 days ago   Up
40 hours   3000/tcp
docker-web-1
2bf04d42af7a   postgres:15-alpine                             "docker-entrypoint.s..."   9 days ago   Up
40 hours (healthy)   5432/tcp
docker-db-1
```

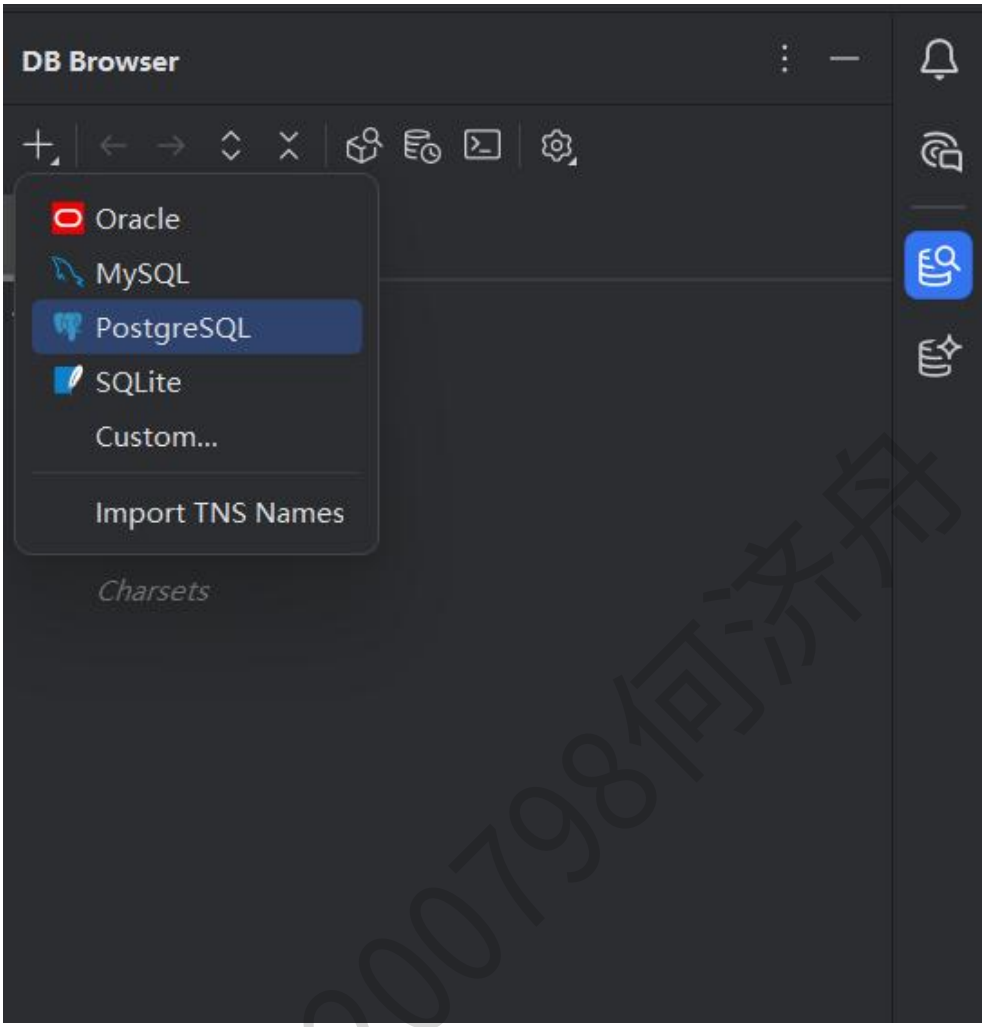
3) pycharm(Python3.8)连接openGauss：

- 下载数据库管理插件 Database Navigator
- 连接openGauss

(1) 连接需要的两个关键信息： IP地址和端口号。 IP地址为一开始认为设置给香橙派的地址。端口号在ssh终端输入 sudo docker ps 查看。找到opengauss那一行的PORTS列。如果显示0.0.0.0:5432->5432/tcp ，则端口是 5432。如果显示0.0.0.0:6600->5432/tcp ，则端口是6600。

(2) 打开右侧栏的DB Browser，点击左上角的‘+’号，选择PostgreSQL。

23009200798何济舟



(3) 填写配置信息

Host (主机): 你的香橙派 IP

Port (端口): 你查到的端口

User (用户): 通常默认是 gaussdb 或者 omm (取决于你启动容器时的设置)

Password (密码): 你在启动 docker 容器时通过 -e GS_PASSWORD=... 设置的密码（此处为Gauss@1234） 。

Database (数据库): 默认通常是 postgres 或 gaussdb。

23009200798何济舟

三、 任务实施

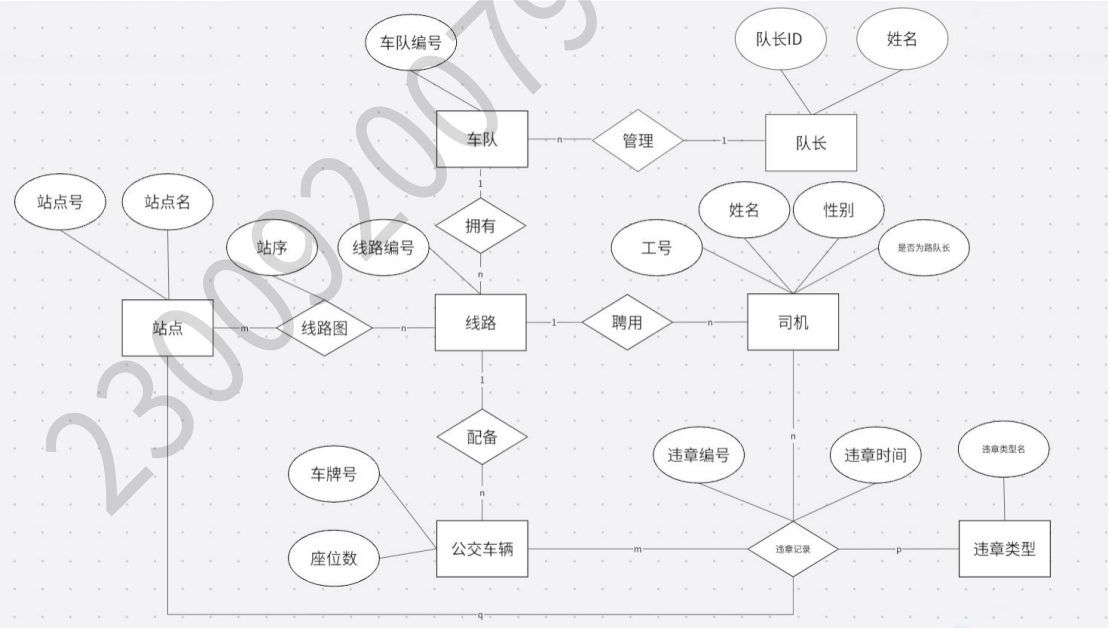
1. 实验要点

1) **E-R图设计与关系模式转换：绘制出E-R图并转换成关系模式，同时注意优化关系模式使其尽可能符合高的范式；**
2) **后台数据库搭建：创建连接池并完善相关操作（取连接、还连接、查询数据、更新修改数据）；在控制台程序connection中完成表的创建；**
3) **业务功能实现：完成实验要求中各功能的代码段；**
4) **前台网页搭建：实现前台网页的搭建；**
5) **实验报告编写。**

2. 关键步骤

步骤一：E-R图设计与关系模式转换。

1) E-R图设计



2) 将E-R图转换成关系模式

加粗部分为主键，斜体为外键

(1) 将实体转换成关系模式

车队表（**车队编号**）

车队长表（**队长ID**，姓名）

线路表（**线路号**）

23009200798何济舟

公交车辆表（**车牌号**，座位数）

司机表（**工号**，姓名，性别，是否为路队长）

站点表（**站点号**，站点名）

违章类型表（**违章类型名称**）

(2) 将联系转换成关系模式

- 1：n联系

对于1：n联系，我们通常不为联系单独建表，而是将“1”端的主键加入到“n”端实体对应的表中，作为外键。

车队表（**车队编号**， *队长ID*）

线路表（**线路号**， *所属车队号*）

车辆表（**车牌号**，座位数， *所属线路号*）

司机表（**工号**，姓名，性别， *所属线路号*）

- m：n联系

对于m：n联系，如果有一个属性是“只有当这两个实体结合在一起时才有意义”，那么这个属性应该直接挂在“联系（菱形）”上。

线路图表（***线路号***， ***站点号***，站序） //线路号+站点号联合构成主键

违章情况表（**违章编号**， 违章司机， 违章车辆， 违章类型， 违章地点，违章时间）

(3) 最终结果

车队表（**车队编号**， *队长ID*）

线路表（**线路号**， *所属车队号*）

车辆表（**车牌号**，座位数， *所属线路号*）

司机表（**工号**，姓名，性别， *所属线路号*，是否为路队长）

违章情况表（**违章编号**， *违章司机*， *违章车辆*， *违章类型*， *违章地点*，违章时间）

站点表（**站点号**，站点名， *所属线路号*）

线路图表（***线路号***， ***站点号***，站序） //线路号+站点号联合构成主键

违章类型表（**违章类型名称**）

步骤二：后台数据库搭建。

1) 创建连接池

考虑到本系统可能会在多台设备上同时使用，如果每次都建立一个链接，操作完之后再销毁，当数量级提升时会极大的影响性能。故此处采用连接池技术，以存储空间换时间。

(1) 任务板块一：配置数据库连接信息

psycopg2需自己提前安装

```
import psycopg2
```

23009200798何济舟

```
from psycopg2 import pool, OperationalError
# 数据库配置信息
DB_CONFIG = {
"host": "_______", # 板子的 IP， 连不上请换 192.168.23.206
"port": "5432", # 默认端口号
"user": "_______", # 用户名
"password": "________", # 密码
"database": "postgres" # 初始连接默认库
}
```

(2) 任务板块二：设计连接池管理器

```
class DBManager:
# 静态变量， 用于存储全局唯一的连接池对象
_connection_pool = None
@staticmethod
def initialize_pool():
"""初始化连接池"""
# 【任务2】 ： 实现连接池初始化
# 【提示】 ： 使用 SimpleConnectionPool 创建池子
if DBManager._connection_pool is None:
try:
DBManager._connection_pool = psycopg2.pool.SimpleConnectionPool(
minconn=1, #(最少保持1个连接)
maxconn=10, #(最多允许10个并发)
**DB_CONFIG
)
print("数据库连接池初始化成功")
except OperationalError as e:
print(f"数据库连接失败: {e}")
raise e
```

(3) 任务板块三：借还连接机制（核心逻辑）

```
@staticmethod
def get_connection():
"""从池中获取一个连接"""
```

23009200798何济舟

```
if DBManager._connection_pool is None:
DBManager.initialize_pool()
# 【任务 3.1】：调用 getconn() 方法从池中取出一个连接
return DBManager._connection_pool.getconn()
@staticmethod
def return_connection(conn):
"""将连接归还给池"""
# 【任务 3.2】：调用 putconn() 方法将 conn 放回池中
# 必须判空，防止归还 None 导致报错
if DBManager._connection_pool and conn:
    DBManager._connection_pool.putconn(conn)
```

(4) 任务板块四：封装通用的 SQL 执行方法

```
@staticmethod
def execute_query(query, params=None):
"""执行 INSERT/UPDATE/DELETE，需要事务提交"""
conn = DBManager.get_connection()
cursor = conn.cursor()
try:
# 【任务 4.1】：执行 SQL 并提交事务
cursor.execute(query, params)
conn.commit() # 提交更改，否则数据库不会保存
print(f"执行成功: {query[:30]}...")
except Exception as e:
# 【任务 4.2】：发生错误时回滚，撤销操作
conn.rollback()
print(f"执行出错: {e}")
finally:
# 【任务 4.3】：无论成功失败，必须关闭游标并归还连接
cursor.close()
DBManager.return_connection(conn)
@staticmethod
def fetch_all(query, params=None):
"""执行 SELECT，返回查询结果"""
```

23009200798何济舟

```
conn = DBManager.get_connection()
cursor = conn.cursor()
result = []
try:
cursor.execute(query, params)
# 【任务 4.4】： 获取所有查询结果
result = cursor.fetchall()
except Exception as e:
print(f"查询出错: {e}")
finally:
cursor.close()
DBManager.return_connection(conn)
return result
```

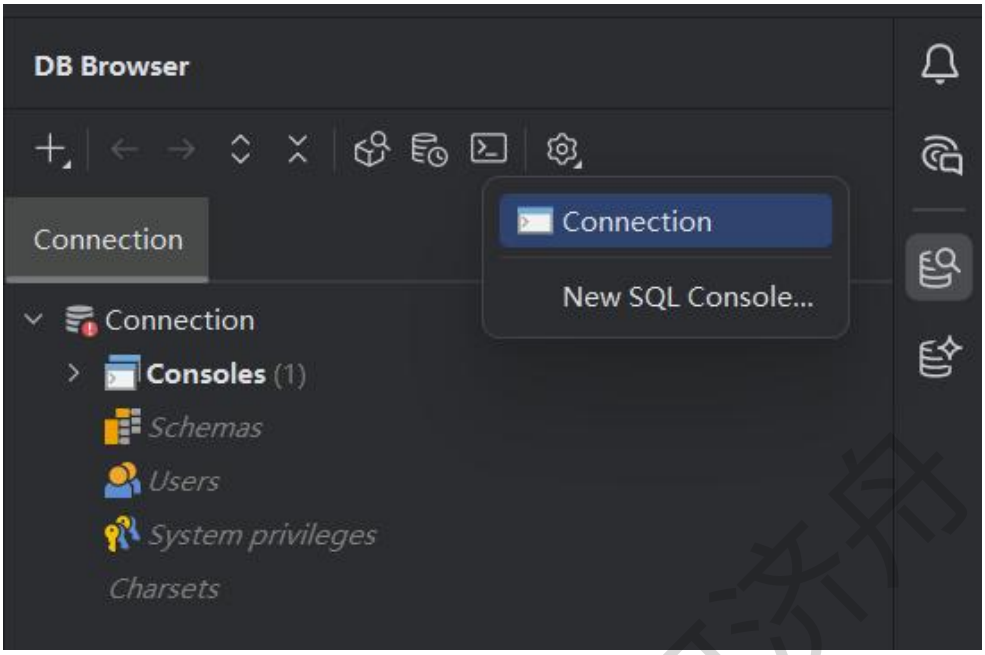
(5) 任务板块五：集成测试

```
if __name__ == "__main__":
try:
# 【任务 5】： 初始化并测试查询
DBManager.initialize_pool()
# 构造一条简单的 SQL： SELECT version();
test_sql = "SELECT version();"
version = DBManager.fetch_all(test_sql)
print(f"数据库连接成功！ 版本信息: {version[0][0]}")
except Exception:
print("连接失败， 请检查 IP、 端口、 密码或防火墙设置。")
```

至此，连接池的创建及相关操作代码段都已编写完成。

2) 在数据库控制台文件 connection 中实现对表结构的定义

23009200798何济舟



(1) 任务板块一：环境初始化

```
# 【任务 1】： 请补充 DROP 语句， 确保脚本可以反复执行而不报错
DROP VIEW IF EXISTS public.v_violation_details CASCADE;
DROP TABLE IF EXISTS public.violation CASCADE;
DROP TABLE IF EXISTS public.bus CASCADE;
DROP TABLE IF EXISTS public.driver CASCADE;
DROP TABLE IF EXISTS public.station CASCADE;
DROP TABLE IF EXISTS public.line CASCADE;
DROP TABLE IF EXISTS public.map CASCADE;
DROP TABLE IF EXISTS public.team CASCADE;
DROP TABLE IF EXISTS public.team_leader CASCADE;
DROP TABLE IF EXISTS public.violation_type CASCADE;
```

(2) 任务板块二：表的构建

```
-- (1) 违章类型表
CREATE TABLE public.violation_type (
type_name VARCHAR(50) PRIMARY KEY
);
-- (2) 车队队长表
CREATE TABLE public.team_leader (
leader_id VARCHAR(20) PRIMARY KEY,
name VARCHAR(50) NOT NULL
```

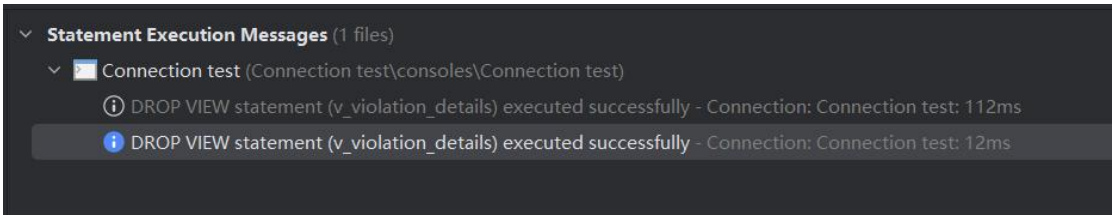
23009200798 何济舟

```
);
-- (3) 车队表
CREATE TABLE public.team (
team_id VARCHAR(20) PRIMARY KEY CHECK (team_id IN ('T01', 'T02', 'T03')),
leader_id VARCHAR(20) NOT NULL,
FOREIGN KEY (leader_id) REFERENCES public.team_leader(leader_id)
);
-- (4) 线路表
CREATE TABLE public.line (
line_id VARCHAR(20) PRIMARY KEY,
team_id VARCHAR(20) NOT NULL,
FOREIGN KEY (team_id) REFERENCES public.team(team_id)
);
-- (5) 站点表
CREATE TABLE public.station (
station_id VARCHAR(20) PRIMARY KEY,
station_name VARCHAR(20) ,
line_id VARCHAR(20) NOT NULL, -- 绑定线路
FOREIGN KEY (line_id) REFERENCES public.line(line_id)
);
-- (6) 线路图表
CREATE TABLE public.map (
station_id VARCHAR(20) ,
line_id VARCHAR(20) ,
station_sequence INT NOT NULL, -- 这里存储“第几站”， 比如 1, 2, 3...
FOREIGN KEY (station_id) REFERENCES public.station(station_id),
FOREIGN KEY (line_id) REFERENCES public.line(line_id),
PRIMARY KEY (station_id,line_id)
);
-- (7) 司机表
CREATE TABLE public.driver (
driver_id VARCHAR(20) PRIMARY KEY,
```

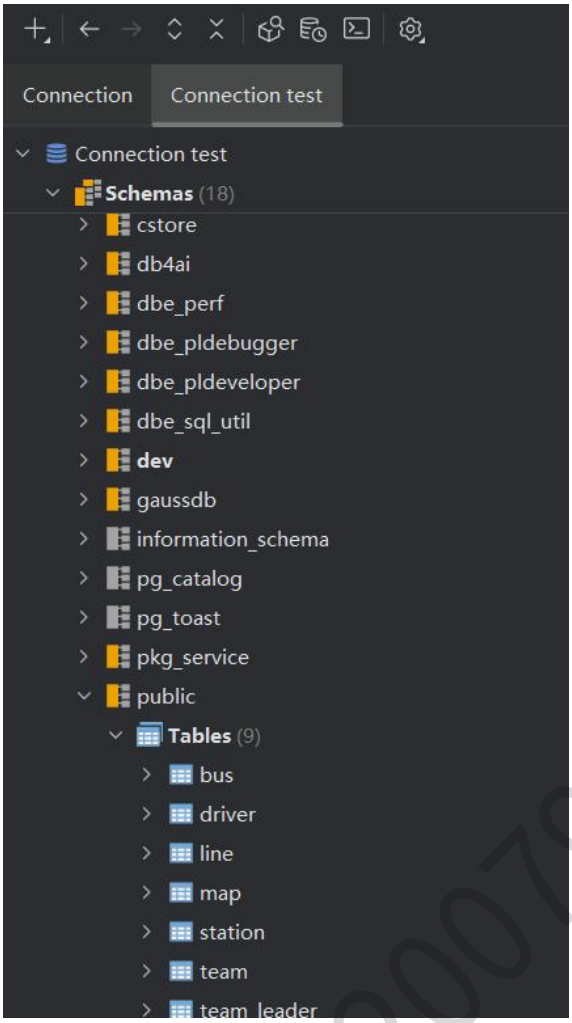
23009200798何济舟

```
name VARCHAR(50) NOT NULL,
gender VARCHAR(10) CHECK (gender IN ('男', '女')),
line_id VARCHAR(20),
-- 添加bool类型， 默认为 false (普通司机)
is_lineleader BOOLEAN DEFAULT FALSE,
FOREIGN KEY (line_id) REFERENCES public.line(line_id)
);
-- (8) 车辆表
CREATE TABLE public.bus (
plate_num VARCHAR(20) PRIMARY KEY,
seats INT,
line_id VARCHAR(20),
FOREIGN KEY (line_id) REFERENCES public.line(line_id)
);
-- (9) 违章记录表
CREATE TABLE public.violation (
v_id SERIAL PRIMARY KEY,
driver_id VARCHAR(20),
plate_num VARCHAR(20),
v_type VARCHAR(50),
location VARCHAR(20),
v_time TIMESTAMP,
FOREIGN KEY (driver_id) REFERENCES public.driver(driver_id),
FOREIGN KEY (plate_num) REFERENCES public.bus(plate_num),
FOREIGN KEY (v_type) REFERENCES public.violation_type(type_name),
FOREIGN KEY (location) REFERENCES public.station(station_name)
);
```

生成成功后需要提交，可以在代码后面补上 COMMIT 语句手动提交，也可以将AUTO COMMIT设置为ON。



23009200798何济舟



步骤三：业务功能实现。

1) 第一步：在main.py中编写业务功能代码

(1) 四个辅助函数

get_line_by_leader(user_id)

作用：身份验证与权限绑定

核心逻辑：根据输入的工号（*user_id*），去 *driver* 表中查找该员工是否为路队长（*is_lineleader = TRUE*）。

用途：在执行“录入违章信息”时调用。它不仅确认了你是否有权录入，还顺带找出了你管理的是哪条线路（*line_id*）。如果返回 *None*，系统就会拒绝操作。

23009200798何济舟

get_stations_by_line(line_id)

作用：获取特定线路的站点清单

核心逻辑：通过多表连接（*station* 表和 *map* 表），按照线路的运行顺序（*station_sequence*）抓取该线路包含的所有站点名称。

用途：用于违章地点的精准选择。它保证了录入的违章地点必须是该线路真实存在的站点，避免了手动输入错别字或虚假地点。

get_db_options(table_name, column_name)

作用：通用的数据库字典读取器

核心逻辑：这是一个非常灵活的函数，可以读取指定表（*table_name*）中的指定列（*column_name*）。

用途：在代码中主要用于获取“违章类型”列表。它从 *violation_type* 表中读取所有预设的违章项目，这样如果以后数据库增加了新的违章类型（比如“抽烟”），代码无需修改就能自动更新选项。

select_from_list(prompt, options)

作用：规范化的人机交互界面

核心逻辑：将传入的列表（*options*）格式化为带数字编号的菜单，并强制要求用户输入有效的编号。

用途：防止用户乱填。配合 *get_db_options* 使用：让用户选“1. 超速”而不是手动打字；配合 *get_stations_by_line* 使用：让用户从该线路的站点里选，提高录入效率和准确性。

(2) add_driver() 与 add_bus()

功能：基础档案录入

add_driver()：录入工号、姓名、性别、所属线路，并能通过 y/n 判断是否授予其“路队长”权限（更新 *is_lineleader* 字段）。

add_bus()：录入车牌号和座位数。它在插入时特别强调了“线路号是否存在”，因为 *bus* 表通常与 *line* 表有关联约束。

(3) add_violation()

功能：违章记录闭环录入（最复杂的业务点）

23009200798何济舟

第一步（鉴权）：调用 *get_line_by_leader* 验证操作人身份。只有路队长能给别人开“罚单”。

第二步（数据校验）：它不允许用户随意填写违章类型和地点，而是强制用户从数据库现有的 *violation_type* 表和该线路关联的 *station* 列表中进行选择。

第三步（格式检查）：通过 *datetime.strptime* 检查用户输入的时间格式是否符合 YYYY-MM-DD HH:MM:SS，防止非法数据导致数据库报错。

(4) query_team_drivers()

功能：跨表查询车队成员

业务逻辑：用户只需输入车队编号（如 T01），系统就能找出该车队下所有线路的所有司机，并清晰地展示其职位（路队长或普通司机）。

(5) query_driver_violations()

功能：精准追溯个人违章

业务逻辑：针对特定司机，查询其在指定时间段内的所有违章详情（时间、类型、地点、当时驾驶的车辆）。

(6) query_team_stats()

功能：管理层的决策支持（聚合统计）

业务逻辑：它不再列出具体哪天谁违章了，而是从宏观角度统计整个车队在某段时间内，各类违章（如“超速”、“闯红灯”）分别出现了多少次。这能帮助车队管理者发现安全隐患的集中点。

这部分代码可以完成实验要求中的操作要求，但是在数据库完全为空（刚建好表）的情况下，单独运行*main.py* 会因为缺少“基础数据”而报错或无法进行。我们需要先运行 *GenerateData.py*（或者手动写 SQL插入）来完成“初始化”。

在*GenerateData.py*中，代码中定义了 3 条线路： L01, L02, L03； 3 个车队： T01, T02, T03。在分配线路归属时，代码用了 *random.choice(team_ids)* 。

GenerateData.py 的逻辑片段:

```
for i, l_id in enumerate(LINES):
t_id = random.choice(team_ids) # 随机选一个车队
```

这是一个完全随机的过程。很有可能 L01, L02, L03 恰好都分给了 T01 和 T03，导致 T02 并没有分到任何线路。

23009200798何济舟

如果后续*GenerateData.py*运行成功，业务功能运行结果如下图所示：

```
 公交安全管理系统
========================================
1. 录入司机基本信息
2. 录入汽车基本信息
3. 录入司机的违章信息(需路队长权限)
4. 查询某车队下的司机信息
5. 查询某司机某时段的违章详情
6. 查询某车队某时段的违章统计
0. 退出系统
========================================
请选择功能 (0-6): 4

[查询车队司机]
请输入车队编号 (如 T01): T03

查询结果 (共 50 人):
工号        姓名        性别    线路      职位
--------------------------------------------------
D002      孙桂珍       女     L01      司机
D003      裴桂英       女     L01      司机
D004      张玲        男     L01      司机
D006      蔡超        女     L01      司机
D007      王平        男     L03      司机
D008      詹磊        男     L02      司机
D010      高健        男     L01      司机
D011      陆俊        男     L03      司机
D012      胡淑兰       男     L03      司机
D013      汤颖        女     L01      司机
```

2) **第二步：在GenerateData.py中编写测试数据生成代码，实现数据库数据“初始化”。**

（1）初始化违章类型（V_TYPES）

循环逻辑：遍历预设的列表 ['闯红灯', '压线行驶', '违章停车', '未礼让斑马线', '超速']。

作用：先执行 SELECT 检查该类型是否已存在，不存在则 INSERT。

目的：确保数据库里的“罪名”字典是完整的，这是后续录入违章的基础。

（2）生成车队及车队长的对应关系

循环逻辑：固定循环 3 次（生成 T01, T02, T03 三个车队）。

作用：随机生成一个名字作为车队长（*team_leader* 表）。创建一个车队并将该队长绑定上去（*team* 表）。

目的：构建管理架构的最顶层。

22009200798何济舟

（3）生成线路、站点及映射关系（Map）

循环逻辑：外层循环 3 条线路（L01-L03），内层循环 7 个站点（A-G）。

作用：

Line表：为每条线路随机分配一个车队。

Station表：创建具体的站点 ID（如 S_L01_A）。

Map表：这是最关键的一步，记录站点在某条线上的先后顺序（*station_sequence*）。

目的：建立公交线路的物理轨迹，后面违章地点的下拉列表就靠这里的数据。

（4）批量生成车辆信息

循环逻辑：循环 20 次。

作用：利用 *fake.license_plate()* 随机生成逼真的车牌号，并随机指派给某条线路（LINES）。

目的：确保系统里有足够多的公交车可以被“驾驶”。

（5）生成司机并任命路队长（分为两个子阶段）

阶段一 ：循环 50 次，生成 50 名司机。此时所有人的 *is_lineleader* 默认为 False（普通司机），并被随机分配到 L01-L03 线路中。

阶段二 ：遍历每条线路，从该线路的司机名单里随机抽取一名“幸运儿”，将其 *is_lineleader* 更新为 True。

目的：模拟真实的人事结构。有了路队长， *main.py* 里的权限验证（功能 3）才有账号可以登录。

（6）随机生成违章历史记录

循环逻辑：循环 30 次。

作用：从已生成的司机、车辆、违章类型中随机“抽签”。

逻辑严谨点：它会通过 *driver_line_map* 找到该司机所属的线路，然后只从该线路的站点中随机选一个作为违章地点。使用 *fake.date_time_this_year()* 生成今年内的随机时间。

目的：填充“黑历史”数据，让你在运行“查询某车队违章统计”时能看到图表或列表数据。

23009200798何济舟

```
F:\PythonProject\BusSystem\.venv\Scripts\python.exe F:\PythonProject\BusSystem\GenerateData.py
 开始生成仿真数据 ...
数据库连接池初始化成功
1. 初始化违章类型...成功
2. 生成车队及队长...成功
3. 生成线路、站点及Map...成功
4. 生成车辆...成功
5. 生成司机并任命路队长...成功
6. 生成违章记录...成功

 仿真数据填充成功!

进程已结束，退出代码为 0
```

至此，我们已经完成了数据库表结构定义、表数据初始化生成填入和实验要求的业务需求。在pycharm中运行main.py可以在控制台验证代码效果。

步骤四：前台网页搭建。

1) 第一步：编写业务逻辑（logic.py）

(1) 系统初始化与权限函数

init_db()

作用：程序的“点火开关”。

逻辑：调用 *DBManager.initialize_pool()* 创建数据库连接池。它是所有后续数据库操作的前提。

check_leader_authority(user_id)

作用：身份卫兵。

逻辑：在 driver 表中根据工号查找，并判断 is_lineleader 标记是否为 TRUE。

(2) 辅助查询函数（数据字典）

get_db_options(table_name, column_name)

作用：下拉列表生成器。

逻辑：传入表名和列名，返回一个简单的列表。

用途：通常用于给前端返回“违章类型”等固定选项。

get_stations_by_line(line_id)

作用：线路地图查询。

逻辑：通过 station 和 map 表的连接查询，按站点顺序排列。

(3) 数据录入函数

add_driver(data)

23009200798何济舟

作用：处理更复杂的人事录入。

add_bus(data)

作用：标准车辆档案录入。

逻辑：将车牌、座位、线路等数据包（data 字典）映射到 bus 表的字段中。

add_violation(data)

作用：违章记录归档。

逻辑：直接执行插入操作。由于 UI 逻辑被剥离，此函数不再负责检查权限，权限检查通常在调用此函数前的 API 拦截层完成。

(4) 数据查询函数

query_team_drivers(team_id)

作用：车队花名册查询。

逻辑：通过司机、线路表的关联，过滤出特定车队的成员。它在返回前会将数据库的布尔值转换为“路队长”或“司机”。

query_driver_violations(driver_id, start_date, end_date)

作用：带时间轴的个人记录查询。

逻辑：使用了倒序排列，让最新的违章记录出现在最前面。同时对时间对象进行了 strftime 格式化，确保返回给前端的是标准的日期字符串。

query_team_stats(team_id, start_date, end_date)

作用：管理层报表生成。

逻辑：核心是 COUNT(*) 和 GROUP BY v_type。它会将该车队在某段时间内所有的违章按类型归类统计，返回一组键值对（例如：[{"v_type": "超速", "count": 5}, ...]）。

2) 第二步：搭建后端 API 服务 (app.py)*//确保已安装 Flask (pip install Flask)*

这份 app.py 是系统的控制层（Controller）。它基于 Flask 框架，将 logic.py 中的业务逻辑封装成 HTTP 接口（RESTful API），使得前端页面（浏览器）能够通过网络请求与数据库进行交互。

23009200798何济舟

(1) 应用初始化与基础路由

logic.init_db()

作用：程序“点火”。

逻辑：在 Flask 应用启动瞬间，立即调用逻辑层的初始化函数，建立数据库连接池，确保后续所有请求都有现成的连接可用。

index()

作用：主页入口。

逻辑：当用户访问根域名（/）时，服务器返回 index.html 页面。它是用户与系统交互的视觉起点。

(2) 基础数据支持接口（这些接口主要负责为前端表单提供“下拉菜单”数据）

get_options(table_name, column_name)

作用：动态获取字典数据。

逻辑：通过 URL 路径参数接收表名和列名，调用 logic.get_db_options 并以 JSON 格式返回结果。例如：访问 /api/options/violation_type/type_name 即可获取所有违章类型。

get_stations(line_id)

作用：获取线路站点详情。

逻辑：根据 URL 传入的线路 ID，返回该线路所有站点的 ID 和名称。这通常用于在录入违章时，根据所选线路动态刷新“违章地点”下拉框。

(3) 权限与数据录入接口（POST）

check_authority()

作用：身份网关。

逻辑：接收前端传来的 JSON 数据（工号），调用 logic.check_leader_authority。前端根据返回的 is_leader 状态来决定是否显示“违章录入”功能。

add_data(type)

作用：通用录入处理器。

逻辑：这是一个“一站式”接口。它根据 URL 中的 type（driver/bus/violation）路由到不同的逻辑函数。

23009200798何济舟

容错机制：使用了 try...except 结构。如果录入失败（如工号重复、外键关联的线路不存在），它会捕获数据库抛出的异常并返回 500 错误码和具体的错误信息给前端。

(4) 数据查询与统计接口（使用 URL 参数（request.args）来接收筛选条件）

query_team_drivers_api()

作用：获取车队成员列表。

逻辑：接收 team_id，返回该车队所有司机的详细信息字典。

query_driver_violations_api()

作用：获取个人违章档案。

逻辑：接收工号、开始日期和结束日期，返回格式化后的违章历史列表。

query_team_stats_api()

作用：生成统计报表数据。

逻辑：接收车队 ID 及时间范围，返回按类型汇总的统计结果。这些数据可以直接被前端图表库（如 ECharts）读取并绘制成饼图或柱状图。

3) 第三步：前端页面（templates/index.html）

(1) 导航与视图切换逻辑（list-group）

功能：充当系统的“主菜单”。

逻辑：通过 data-target 属性标识每个按钮对应的功能区 ID。

用途：配合 JavaScript（通常在 script.js 中），实现点击左侧菜单时，右侧对应 feature-section 的显示与隐藏。这让系统在一个页面内就能完成所有操作，无需频繁刷新。

(2) 录入司机信息模块（section-add-driver）

对应函数：后端 logic.add_driver() 和 app.py 的 /api/add/driver 接口。

职位任命：下拉框将 true/false 传递给后端，后端 logic.py 会负责将其转换为数据库布尔值。

表单约束：使用 required 属性确保工号、姓名等关键数据不会为空。

(3) 录入车辆信息模块（section-add-bus）

对应函数：后端 *logic.add_bus()* 和 *app.py 的 /api/add/bus* 接口。

23009200798何济舟

逻辑：接收车牌号、座位数和所属线路。

用途：录入车辆档案。录入时，后端会检查线路号是否存在，以维持数据库的参照完整性。

(4) 权限验证模块（section-violation-auth）

对应函数：后端 *logic.check_leader_authority()* 和 app.py 的 */api/check_authority* 接口。

业务逻辑：这是系统的“门禁”。用户输入工号后，前端请求后端。

只有当返回 is_leader: True 时，才会自动切换到下一个“录入违章信息”界面。

它还会返回该队长管理的线路 ID，用于后续过滤站点。

(5) 录入违章信息模块（section-add-violation）

对应函数：后端 *logic.add_violation()* 及辅助函数 *get_stations_by_line()*。

核心逻辑：当此模块激活时，前端会调用 /api/options/violation_type/type_name 加载违章类型，并根据该队长管理的线路 ID 调用 /api/stations/<line_id> 加载站点。

数据隔离：队长只能在下拉菜单中选择自己管辖线路的站点，避免了越权录入或数据填错。

(6) 查询车队司机模块（section-query-team-drivers）

对应函数：后端 *logic.query_team_drivers()* 和 app.py 的 */api/query/team_drivers* 接口。

展示逻辑：前端接收到 JSON 数组后，循环生成 <tr> 标签插入到 *tbody-team-drivers* 中。

交互点：它将数据库中的布尔值显示为直观的“路队长”或“司机”文字。

(7) 查询司机违章详情模块（section-query-driver-violations）

对应函数：后端 *logic.query_driver_violations()*。

逻辑：提供工号和日期筛选范围。

(8) 查询车队违章统计模块（section-query-team-stats）

对应函数：后端 *logic.query_team_stats()*。

结果呈现：不再使用表格，而是使用 *list-group* 列表组来展示每种违章类型的发生次数。

用途：为管理者提供车队整体违章分布的概览。

23009200798 何济舟

4) 第四步：前端交互逻辑（static/script.js）

(1) 基础 UI 工具函数

showAlert(message, type)

作用：全局通知弹窗。

逻辑：根据传入的 type（如 success 绿色、danger 红色）修改 Bootstrap 警告框的样式。

switchSection(targetId)

作用：实现无刷新的功能切换。

逻辑：通过操作 CSS 类名（active），隐藏当前显示的 feature-section 并显示目标区域，同时高亮左侧对应的菜单项。

(2) 通用数据录入逻辑

bindFormSubmit(formId, apiEndpoint)

作用：表单提交的“万能胶水”。

逻辑：拦截表单的默认提交行为（阻止页面刷新）；使用 FormData 自动提取输入框里的数据，打包成 JSON；使用 fetch 发送到指定的后端 API（如 /api/add/driver）；根据后端返回的状态码（200 或 500）调用 showAlert 反馈结果。

(3) 特殊业务交互逻辑

[功能 3] 权限验证与级联加载

交互逻辑：当用户点击“验证身份”时，先调用权限检查接口。验证成功后，不仅切换界面，还会并行触发两个新请求（违章类型、专属站点）。使用 Promise.all 确保所有下拉菜单数据都加载完后，再更新界面。

目的：确保路队长只能选择自己管辖线路的站点，实现了前端的数据预过滤。

[功能 4/5/6] 数据查询与表格渲染

逻辑流：读取输入条件 → 发送 GET 请求（带参数） → 循环遍历返回的 JSON 数组。

渲染技术：使用模板字符串（${...}）动态生成 HTML 片段（如 <tr> 或 <li>），然后通过 innerHTML 注入到页面的表格或列表中。

职位高亮：在查询车队司机时，通过条件判断，专门给“路队长”打上一个鲜艳的标签。

23009200798何济舟

(4) 自动化初始化工作

日期自动填充逻辑

作用：提升用户体验。

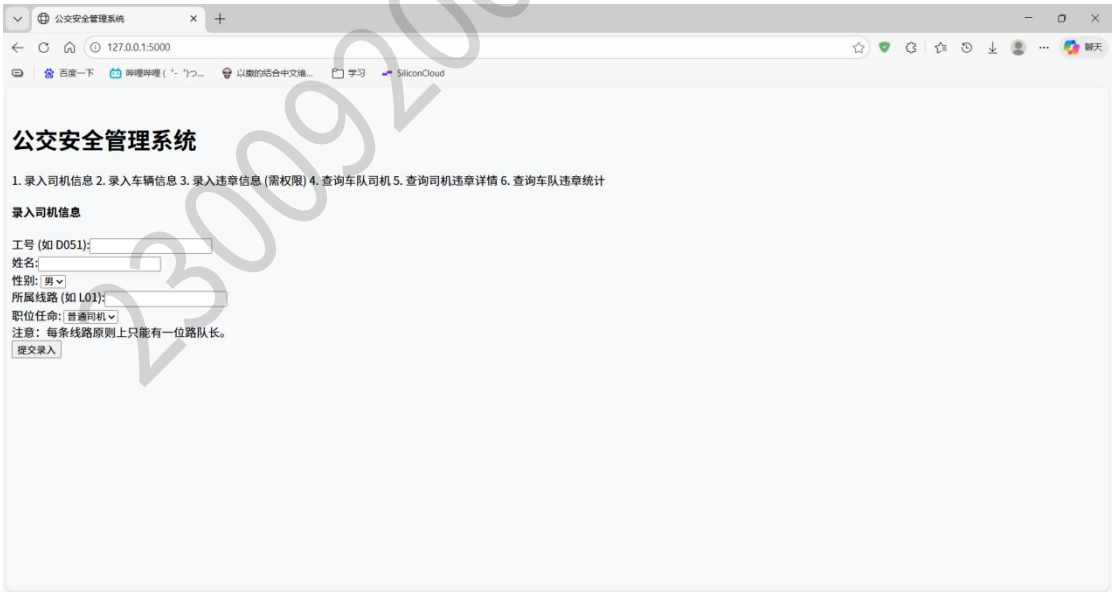
逻辑：在页面加载完成（DOMContentLoaded）后，自动计算当前月份的第一天和最后一天，并填入查询窗口的日期控件中。

目的：让用户一打开查询界面，默认就能看到本月的统计数据，减少手动输入的麻烦。

运行操作：

1. 确保已安装 Flask：在终端执行 pip install Flask。
2. 运行 app.py。控制台会显示 Running on http://0.0.0.0:5000。

3. 打开浏览器，访问 http://127.0.0.1:5000。



步骤五：实验报告编写。

1) 大模型说明

大模型的名称及版本，用于后续的哪些部分，并且在后续的每个部分展示大模型的使用方法。

23009200798何济舟

2) 需求分析（系统数据和功能）

数据字典。

3) 概念结构设计（E-R图设计）

ER图需要与数据库语义一致，可用Erwin，可使用多种大模型生成E-R图。

4) 逻辑结构设计（E-R图转换为关系模型）

关系模型给出表名，列名，列类型，主码，外码，其他约束，进行范式分析。索引、视图、存储过程等。

5) 程序开发环境及应用环境

列出操作系统、编程语言、开发工具。

6) 应用程序设计中遇到的问题及解决方法

数据库设计中的问题，程序设计中的问题。

7) 总结

课程、实验收获，课程、实验建议等。

8) 附录：建立数据库和应用程序的主要代码

只列出主要代码，需要有注释。

2300920079 8何济舟

四、 任务拓展

本公交安全管理系统可以参考实际的西安市公交安全运营管理系统进行拓展。

1. 违章视频管理：为每条违章记录关联视频证据，支持违章图片、视频文件上传、存储。

2. 处罚管理：根据违章类型自动匹配处罚规则扣分、罚款、停驾等，生成处罚通知单并跟踪处理状态。

3. 培训管理：针对累计扣分达阈值的司机，自动生成培训计划并记录培训考勤与考核结果。

4. 统计评价：提供违章多维统计看板，包含视频取证率、处罚执行率、培训通过率等。

5. 违章地图可视化：将违章地点在电子地图上展示，支持按时间/类型聚类显示及热力图分析。

2300920079 8何济舟

五、 实验总结

通过本次基于香橙派鲲鹏 Pro 开发板与 openGauss 的公交安全管理系统实验，完成了以下关键学习目标：

1. 深入理解国产软硬件平台的全栈部署方法：在香橙派鲲鹏 Pro 开发板上完成 openGauss 数据库与 Web 应用的部署、配置及集成，熟悉国产化开发环境。

2. 熟练掌握 openGauss 数据库完整性与性能优化：在 openGauss 中定义各类数据完整性约束，运用索引、视图、存储过程等高级 SQL 特性来优化查询与简化逻辑。

3. 熟练掌握基于规范化理论的数据库设计方法：运用规范化理论设计低冗余、结构清晰的数据库模式，使用专业设计工具完成从概念模型到物理模型的构建。

4. 全面掌握大模型辅助高效开发方法：有效利用大模型生成符合业务需求的核心代码片段，并将其整合调试，提升数据库应用开发的效率与实践能力。

5. 熟练掌握数据库应用系统全流程开发方法：掌握从后端业务逻辑、前端界面到数据库访问的完整开发流程，能够构建一个功能完备的信息管理系统。

2300920079 何沂书